

Principles of Cloud and DevOps Engineering – Project

Part 1

Gabriel Alvarez Arzate – 3017178

1. Introduction

This document outlines the first practical component of the project for the course *Principles of Cloud Engineering and DevOps*. The objective of this initial phase is to define and implement a cloud environment on Microsoft Azure, utilizing the Infrastructure as Code (IaC) methodology covered in class.

We are building this infrastructure with the specific purpose of ensuring that, during the subsequent development phases of the project, it can support a website capable of storing and displaying images; in this first part, the focus centers on correctly defining, connecting, and deploying these resources in a reproducible and automated manner.

2. Prerequisites:

- Azure Account.
- Azure CLI – for connecting to Azure via the CLI.
- Terraform – for infrastructure creation.
- VS Code – recommended editor with the Terraform extension for file management (optional).

3. How to run:

- First, we need to sign in to Azure from the terminal to access Azure resources and retrieve the `subscription\_id`:
 - **az login**
- After logging in, run `az account show` to retrieve your `subscription_id`. Copy the `id` field and paste it into `terraform.tfvars`.
 - **az account show**
- Next, we must create the entire structure of our project using Terraform—specifically, the structure that will be outlined in the documentation later on. Once the project structure is in place and all file contents (the code) have been added, we need to execute the following command to initialize Terraform. This command will download the defined providers, lock in the exact versions specified in your configuration files, and finally, set up the project's backend—which, in our case, will be configured locally:
 - **terraform init**

- The following command is used when we wish to make changes to our Azure environment, but only after performing a preliminary verification to see exactly what Terraform will do before actually carrying it out. It is recommended to use this command to ensure that the resulting behavior aligns with our plan.

— **terraform plan**

- Subsequently, we execute this command to apply the new changes to Azure once we have confirmed them; you must type "yes" when prompted for confirmation to allow the program to proceed with the execution.

— **terraform apply**

4. Approach & Decisions:

4.1 Code Structure

- **providers.tf** — Declares the cloud provider Terraform will work with (in this case Azure) and the required version. It functions similarly to an import statement in a programming language.
- **variables.tf** — Defines the configurable parameters of the project, such as the resource names, region, and subscription ID. This avoids hardcoded values scattered throughout the code and makes the configuration reusable.
- **main.tf** — Contains the actual resource definitions: the Resource Group, Storage Account, Key Vault, App Service Plan, and App Service. This is the core of the project.
- **outputs.tf** — Specifies which information Terraform should display after a successful deployment, such as the App Service URL or the Key Vault URI.
- **terraform.tfvars** — Stores sensitive or environment-specific values such as the subscription_id. This file is excluded from version control via .gitignore to prevent exposing credentials, while still allowing Terraform to read the values at runtime.

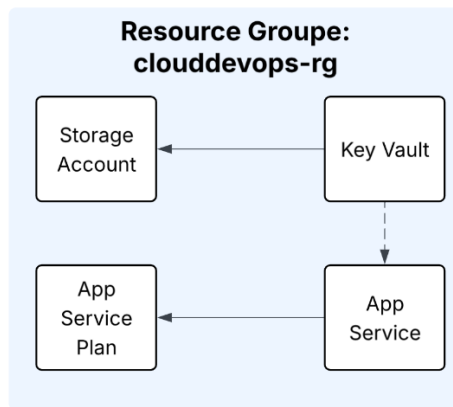
4.2 Design Decisions

- **Region:** For this project, I decided to use the "germanywestcentral" region because, among the regions available to my student account, it was the geographically closest to my location. This decision was made after attempting to deploy resources in "westeurope," which was blocked by a policy within the student subscription.
- **Use of `random\_string`:** The sole purpose of using `random\_string` is to prevent naming conflicts within the Azure environment, given that certain resources—such as Storage Accounts—require unique names. I set its length to 6 characters to further reduce the probability of it clashing with any existing data.

- **File Separation:** The project files are organized according to their specific responsibilities, adhering to standard Terraform conventions and industry best practices. This separation makes the code easier to maintain and understand, as each file serves a clear and specific purpose.
- **Use of LRS:** I opted for Locally Redundant Storage (LRS) due to its data replication capabilities and because it represents the most cost-effective option available with my student account—an option that is, moreover, entirely sufficient for the development of this project.

5. Resources & Connections

Diagram:



Resource Group:

This is the logical container that groups together all resources related to a project—specifically, it houses the Storage Account, Key Vault, and App Service.

It offers several advantages: it allows for more effective collective cost management of resources; it enables the deletion of all associated resources simply by deleting the group; and it aids in overall project organization.

Key Vault & Storage Account connection:

The Key Vault and Storage Account are not directly connected at this stage of the project. However, they are designed to work together in the next phase: the Storage Account generates sensitive credentials (access keys and connection strings) that are needed by the application to read and write files. Instead of hardcoding these credentials in the application code, they will be stored securely in the Key Vault. The App Service will then retrieve them at runtime, ensuring that no sensitive data is exposed in the codebase.

App Service & App Service Plan:

These are two services that work in tandem for the following reasons:

App Service Plan: This defines the underlying hardware to be utilized—specifically, the allocated CPU, RAM, and the associated cost of usage. In essence, we are defining the virtual server that the resources will run on.

App Service: This is the environment where the application itself will execute atop the previously created plan; that is, the application we will build later on will run within this specific plan.

Implicit dependencies in Terraform:

All resources reference each other through Terraform's dependency system rather than using hardcoded values. For example, instead of writing "germanywestcentral" directly in each resource, the Storage Account and Key Vault reference the Resource Group's location using `azurerm_resource_group.main.location`. This creates an implicit dependency, Terraform automatically determines the correct creation order, ensuring the Resource Group exists before any other resource is deployed.

6. Authentication/ Identity Context

Terraform authenticates with Azure using the active session established via ``az login``. This session provides the necessary permissions to create and manage resources. The ``subscription_id`` is used to specify which Azure subscription the resources should be deployed to.

Key Vault requires explicit access policies, as, by default, no one has access to it not even the creator. Access is granted by specifying a ``tenant_id`` and an ``object_id``, both of which are obtained dynamically using the ``azurerm_client_config`` data source. In this project, the current user has been granted read, list, set, delete, and purge permissions on secrets, enabling full management of the sensitive data stored within the vault.